Sequential Digital Circuits in VHDL on the DE1-SoC FPGA

Digital Systems Electronics Course

Erfan Taheri
DET / Politecnico di Torino
Erfan.Taheri@studenti.polito.it

Abstract

This report covers the design, simulation, and synthesis of five sequential digital systems implemented in VHDL and targeting the DE1-SoC FPGA board. Starting from a structural gated SR latch, the work progresses through two versions of a 16-bit synchronous counter (structural and behavioral), a digit flasher with clock division, and finally an FSM-based reaction timer with BCD display output. All designs were simulated in ModelSim and synthesized in Quartus Prime, with waveform inspection and RTL analysis used throughout for verification.

Contents

1 Introduction **3**

2 Tools and Target Platform **3**

3 Project Overview **3**

4 Part 1: Gated SR Latch **3**
4.1 Design Description 3
4.2 Simulation 4
4.3 Results 5

5 Part 2: Structural 16-bit Synchronous Counter **5**
5.1 Design Description 5
5.2 Board Interface 5
5.3 Results 7

6 Part 3: Behavioral 16-bit Synchronous Counter **8**
6.1 Design Description 8
6.2 Structural vs. Behavioral Comparison 9
6.3 Discussion 9

7 Part 4: Flashing Digits 0–9 **9**
7.1 Design Description 9
7.2 Simulation Note 10
7.3 Results 12

8 Part 5: Reaction Timer **12**
8.1 Design Description 12
8.2 FSM Design 12
8.3 Timing 13
8.4 Results 15

9 Verification Summary **15**

10 Conclusion **16**

1 Introduction

The aim of this lab was to design and verify a set of sequential digital circuits in VHDL, progressing from a simple latch to a complete timing system. Working on the DE1-SoC FPGA gave a practical context for each design, since the board's switches, push-buttons, LEDs, and seven-segment displays served as the primary I/O.

The work covered structural and behavioral description styles, reusable component design, clock division, FSM-based control, and testbench-driven verification. Alongside writing the VHDL, a key goal was to develop intuition for reading ModelSim waveforms and Quartus RTL diagrams to confirm that the synthesized hardware matched the intended design.

2 Tools and Target Platform

Table 1: Development tools and target hardware

Item	**Description**
Hardware platform	DE1-SoC FPGA board
Hardware description language	VHDL
Simulation tool	ModelSim
Synthesis tool	Quartus Prime
Main clock	50 MHz board clock
Inputs	Switches and push-buttons
Outputs	LEDs and seven-segment displays

3 Project Overview

The project was divided into five parts, each building on the previous one.

Table 2: Summary of implemented designs

Part	**Design**	**Key concept**
1	Gated SR latch	Gate-level structural description and latch simulation
2	16-bit synchronous counter (structural)	T flip-flop chain with synchronous carry logic
3	16-bit synchronous counter (behavioral)	Single-process arithmetic increment
4	Digit flasher	Clock division and modulo-10 counting
5	Reaction timer	FSM control, millisecond timing, BCD display output

4 Part 1: Gated SR Latch

4.1 Design Description

The first circuit is a gated SR latch written in structural VHDL. The enable signal (labeled `Clk` in the entity) gates the Set and Reset inputs before they reach the cross-coupled NOR latch core. Four intermediate signals—`R_g`, `S_g`, `Qa`, and `Qb`—make the gate-level structure explicit. A Quartus `keep` attribute is applied to these signals so the synthesizer does not collapse them

away, which would otherwise prevent the intended structure from appearing in the RTL and technology viewers.

4.2 Simulation

A dedicated testbench exercised the four fundamental latch conditions in sequence: reset (`R=1, S=0`), set (`R=0, S=1`), hold (`R=0, S=0`), and the forbidden state (`R=1, S=1`). A fifth vector returns the latch to a known valid state after the invalid input. The clock runs at 50 MHz (20 ns period) and the stimulus changes at one-period intervals so each case is fully captured in the waveform.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY SRlatch IS
5      PORT ( Clk, R, S : IN STD_LOGIC;
6             Q : OUT STD_LOGIC);
7  END SRlatch;
8
9  ARCHITECTURE Structural OF SRlatch IS
10     SIGNAL R_g, S_g, Qa, Qb : STD_LOGIC;
11     ATTRIBUTE keep : boolean;
12     ATTRIBUTE keep of R_g, S_g, Qa, Qb : SIGNAL IS true;
13 BEGIN
14     R_g <= R AND Clk;
15     S_g <= S AND Clk;
16     Qa <= NOT (R_g OR Qb);
17     Qb <= NOT (S_g OR Qa);
18     Q <= Qa;
19 END Structural;
20
```

Figure 1: Structural VHDL source for the gated SR latch.

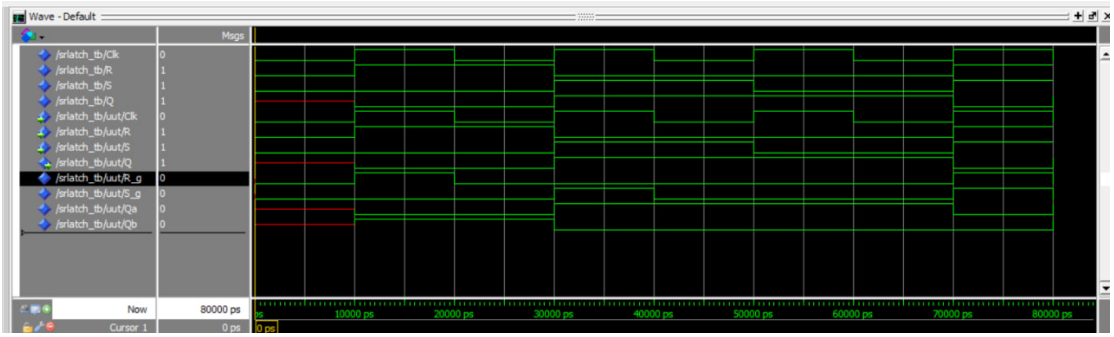


Figure 2: ModelSim waveform for the gated SR latch testbench.

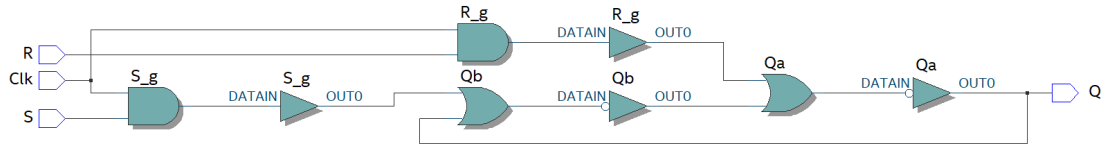


Figure 3: RTL Viewer showing the preserved gate-level structure.

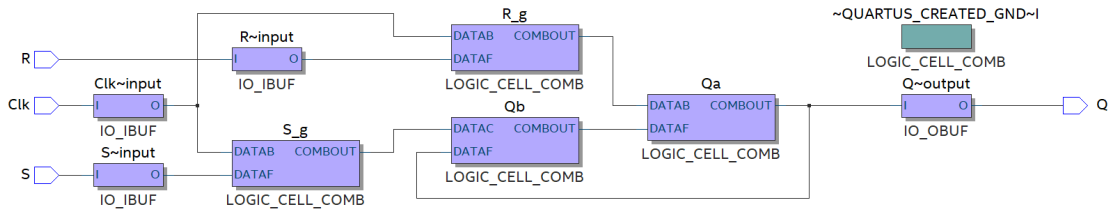


Figure 4: Technology Map Viewer result for the gated SR latch.

4.3 Results

The simulation matched the expected truth table. While the enable signal was low, the output held its previous value regardless of S and R. When enabled, set and reset operated normally. In the forbidden state both complementary nodes were driven to the same logic level, producing the ambiguous output characteristic of an SR latch. The RTL viewer confirmed that the `keep` attribute successfully preserved the four-gate structure.

5 Part 2: Structural 16-bit Synchronous Counter

5.1 Design Description

This part implements a 16-bit synchronous counter by connecting sixteen instances of a reusable T flip-flop component. The T flip-flop has an active-low asynchronous clear and toggles its state on the rising clock edge whenever the toggle input is asserted:

$$Q_{\text{next}} = Q \oplus T$$

The first flip-flop's toggle input is the global enable signal. Each subsequent flip-flop toggles only when the enable is active *and* all lower-order bits are high, forming a synchronous carry chain:

$$\texttt{carry}[i] = \texttt{enable} \cdot \prod_{k=0}^{i} Q[k]$$

This ensures all flip-flops that need to change do so on the same clock edge, which is why the design is synchronous rather than ripple-carry. The VHDL uses a `for generate` statement to instantiate flip-flops 1–15 and their carry signals, keeping the code concise despite the 16-bit width.

5.2 Board Interface

Table 3: Board interface for the structural counter

Signal	Function
`KEY(0)`	Manual clock pulse
`SW(0)`	Active-high reset (inverted internally)
`SW(1)`	Count enable
`HEX0--HEX3`	Counter value in hexadecimal

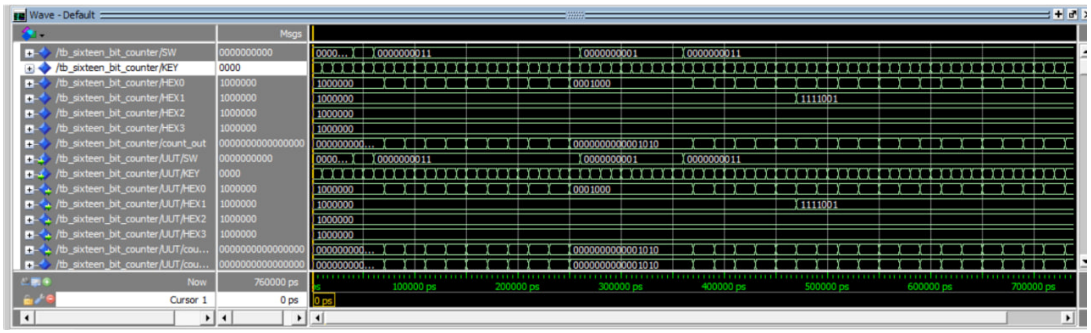
The 16-bit output is split into four 4-bit nibbles, each decoded by a `seven_seg_dec` component and routed to one display.

```vhdl
ENTITY Tflip_flop IS
    PORT (
        clk    : IN  std_logic;  -- Clock input
        clear  : IN  std_logic;  -- Active-low asynchronous clear
        toggle : IN  std_logic;  -- Toggle control
        q_out  : OUT std_logic   -- Output
    );
END Tflip_flop;

ARCHITECTURE behavior OF Tflip_flop IS
    SIGNAL q_temp : std_logic := '0';  -- Internal state
BEGIN
    PROCESS (clk, clear)
    BEGIN
        IF clear = '1' THEN
            q_temp <= '0';  -- Asynchronous clear
        ELSIF rising_edge(clk) THEN
            q_temp <= toggle XOR q_temp;  -- Toggle behavior
        END IF;
    END PROCESS;

    q_out <= q_temp;
END behavior;

```

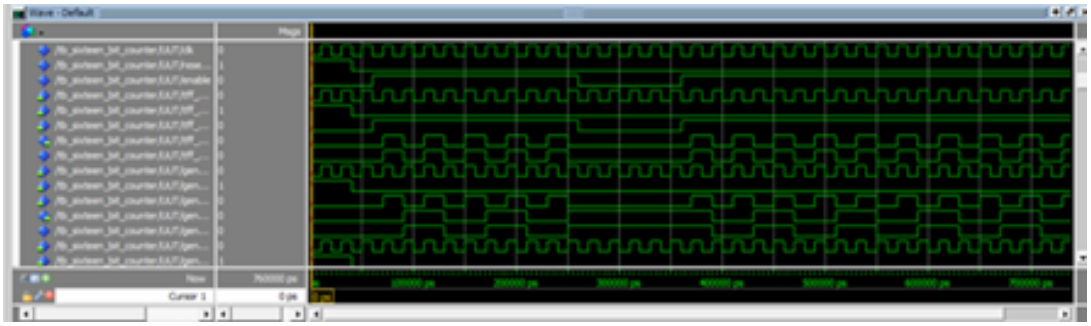
Figure 5: VHDL source of the T flip-flop component.

```vhdl
BEGIN
    -- Control signal mapping
    clk      <= KEY(0);
    enable   <= SW(1);
    reset_n  <= NOT SW(0);  -- SW(0) is active-high; convert to active-low

    -- First T flip-flop
    tff_0 : Tflip_flop
        PORT MAP (
            clk    => clk,
            clear  => reset_n,
            toggle => enable,
            q_out  => counter_value(0)
        );

    -- Carry signal for second stage
    carry_chain(0) <= enable AND counter_value(0);

    -- Generate T flip-flops from bit 1 to 15
    gen_counter : FOR i IN 1 TO 15 GENERATE
        tff : Tflip_flop
            PORT MAP (
                clk    => clk,
                clear  => reset_n,
                toggle => carry_chain(i - 1),
                q_out  => counter_value(i)
            );

        -- Generate carry chain for i < 15
        carry_gen : IF i < 15 GENERATE
            carry_chain(i) <= carry_chain(i - 1) AND counter_value(i);
        END GENERATE carry_gen;
    END GENERATE gen_counter;

    -- Output assignment
    count_out <= counter_value;
```

Figure 6: Structural 16-bit counter connecting sixteen T flip-flop instances.



(a) Full simulation waveform.



(b) Zoomed view showing individual bit transitions.

Figure 7: ModelSim waveform for the structural 16-bit counter.

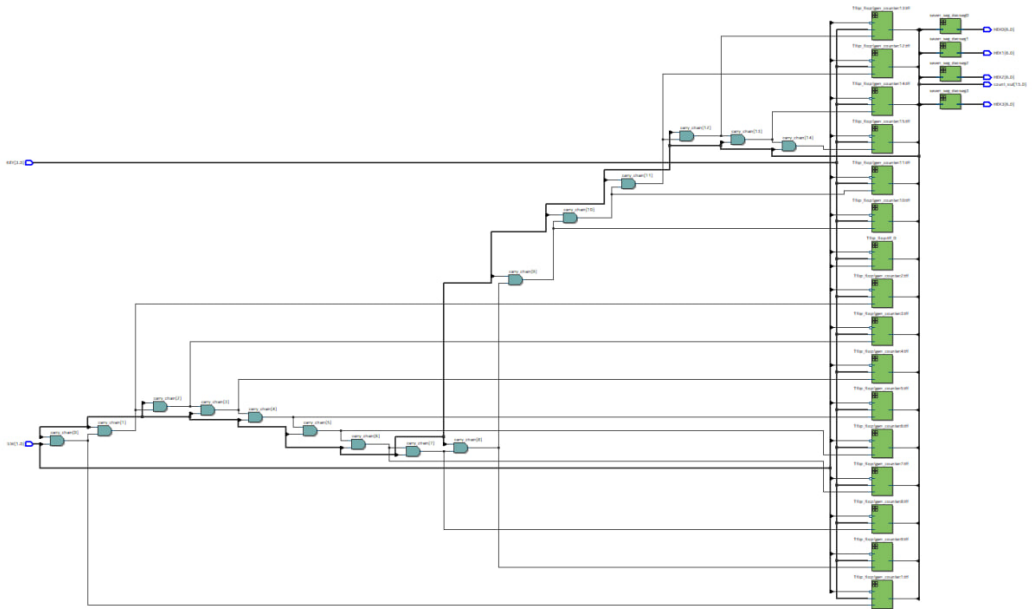


Figure 8: RTL Viewer for the structural 16-bit counter, showing the flip-flop chain and carry logic.

5.3 Results

The waveform confirms that the counter increments on every rising clock edge when enable is high, holds its value when enable is deasserted, and clears to zero asynchronously when reset is applied. The RTL diagram shows the expected chain of T flip-flops with AND-based carry propagation between stages.

6 Part 3: Behavioral 16-bit Synchronous Counter

6.1 Design Description

Part 3 re-implements the same 16-bit counter using a behavioral style. Instead of wiring flip-flop instances by hand, a single process operates on an `unsigned` register and the synthesis tool infers all required flip-flops and increment logic automatically:

```
process(clock, reset_n)
begin
    if reset_n = '0' then
        count_reg <= (others => '0');
    elsif rising_edge(clock) then
        if enable = '1' then
            count_reg <= count_reg + 1;
        end if;
    end if;
end process;
```

Listing 1: Core behavioral counter process

The result is functionally identical to Part 2 but considerably shorter and easier to maintain.

```
25  begin
26
27      -- Synchronous Process
28      process (KEY(0), SW(0))
29      begin
30          -- Active-low reset
31          if (SW(0) = '0') then
32              count_reg <= (others => '0');
33          elsif rising_edge(KEY(0)) then
34              if SW(1) = '1' then
35                  count_reg <= count_reg + 1;
36              end if;
37          end if;
38      end process;
39
40      -- Output assignment
41      count_out <= std_logic_vector(count_reg);
```

Figure 9: Behavioral VHDL source for the 16-bit counter.

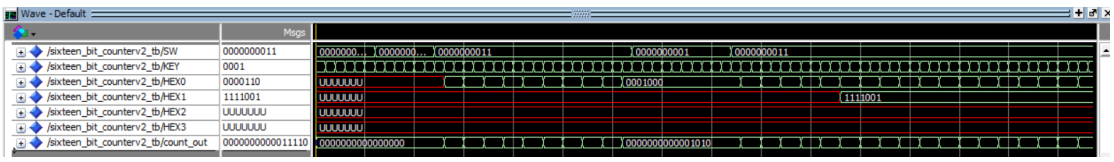


Figure 10: ModelSim waveform for the behavioral counter.

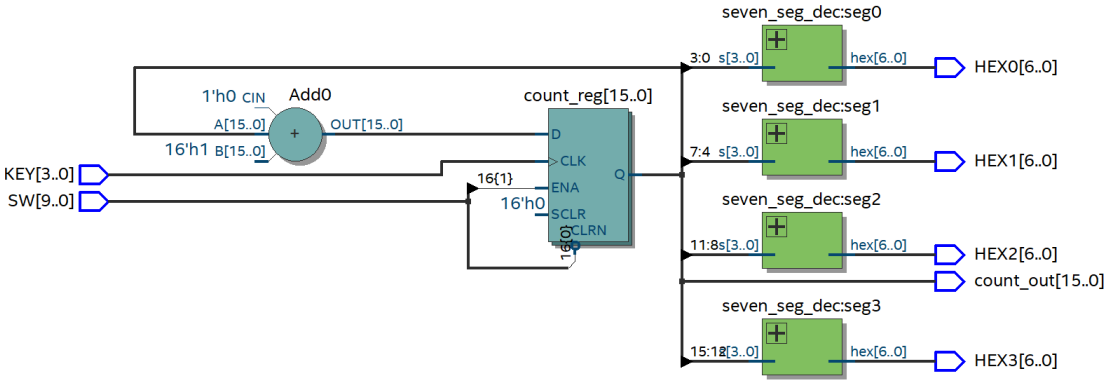


Figure 11: RTL Viewer for the behavioral counter. The synthesizer produces an equivalent register-adder structure.

6.2 Structural vs. Behavioral Comparison

Table 4: Structural vs. behavioral counter

Aspect	Structural	Behavioral
Description style	Explicit flip-flop instances	Single arithmetic process
Code length	Longer; wiring is manual	Compact
Hardware visibility	Directly visible in RTL	Inferred by synthesizer
Wiring errors	Possible if carry logic is wrong	Not applicable
Preferred for	Teaching hardware internals	Practical FPGA development

6.3 Discussion

Both implementations produce the same observable behavior. The structural approach is valuable when learning, because every flip-flop and carry wire is explicit. The behavioral approach is preferable in practice: it is less error-prone, shorter, and more readable, especially as counter widths grow.

7 Part 4: Flashing Digits 0–9

7.1 Design Description

This part drives a single seven-segment display through the digits 0–9 in sequence, changing once per second. The design connects two counter stages. The first is a 26-bit counter that divides the 50 MHz board clock down to a 1 Hz tick:

$$50 \times 10^6 \text{ cycles} \xrightarrow{\div\, 50{,}000{,}000} 1 \text{ Hz}$$

The tick resets the divider and advances the second stage, a 4-bit digit counter. When the digit counter reaches 10 (binary `1010`), it resets to 0, creating a modulo-10 cycle. The `seven_seg_dec` component converts the 4-bit value to a seven-segment code for `HEX0`.

Both stages use the same T flip-flop component from Part 2, so the whole design stays structural and consistent with the earlier parts.

7.2 Simulation Note

Waiting for 50 million clock cycles in simulation would produce an impractically long waveform. The simulation version therefore sets the divider terminal count to 50 instead of 50,000,000. The logic is otherwise unchanged, so the simulation faithfully captures the tick-and-advance behavior.

```
84      process(CLOCK_50)
85      begin
86          if rising_edge(CLOCK_50) then
87              if one_hz_tick = '1' then
88                  reset_next <= '1';
89              else
90                  reset_next <= '0';
91              end if;
92          end if;
93      end process;
```

Figure 12: VHDL source for the digit flasher.

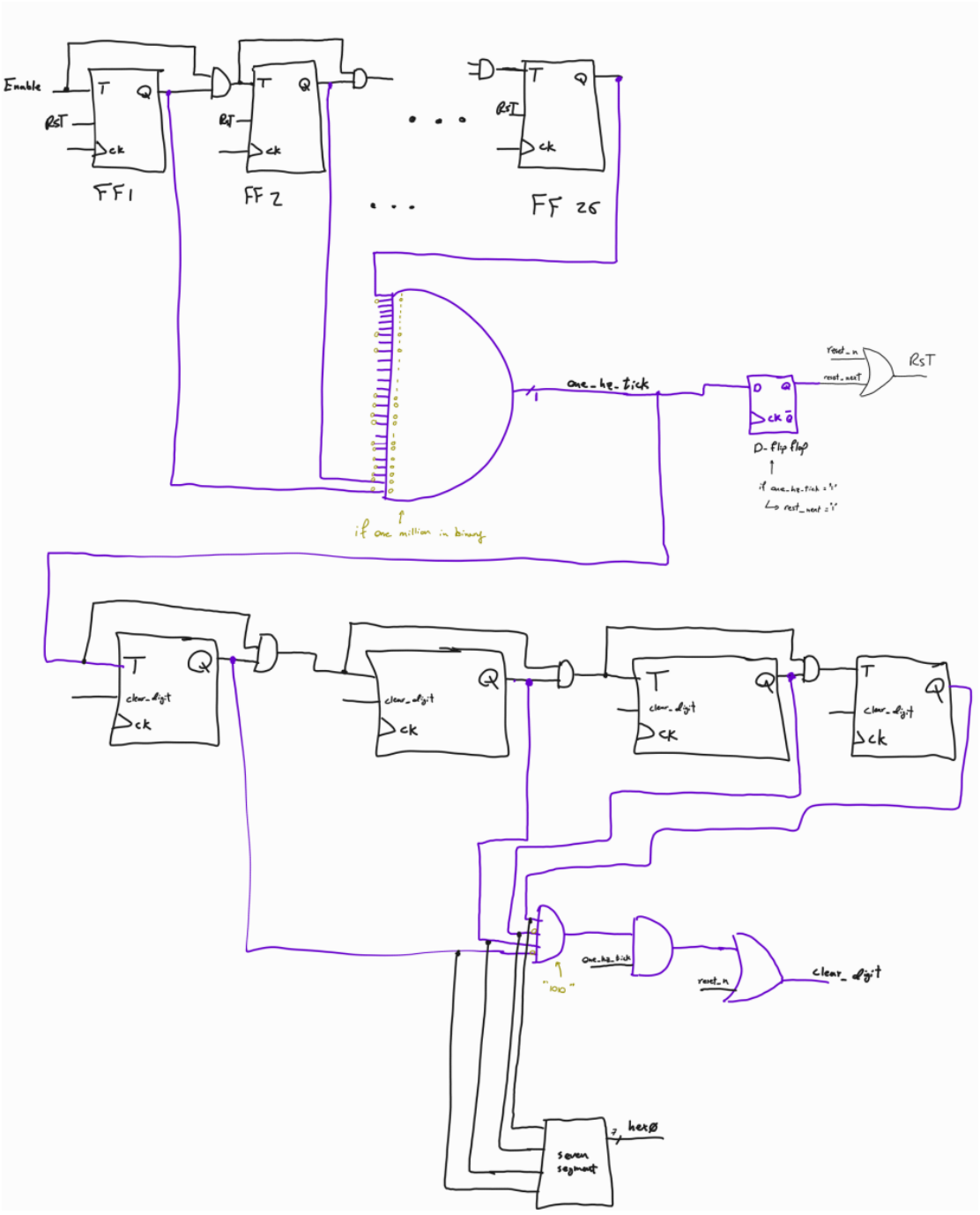


Figure 13: Block diagram of the digit flasher showing the two-stage counter chain.

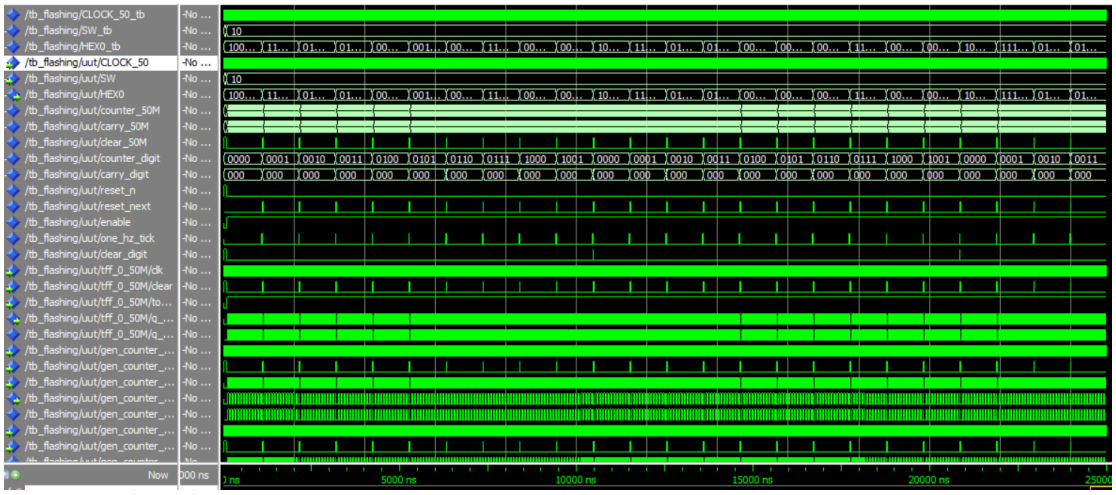


Figure 14: Simulation overview waveform for the digit flasher.

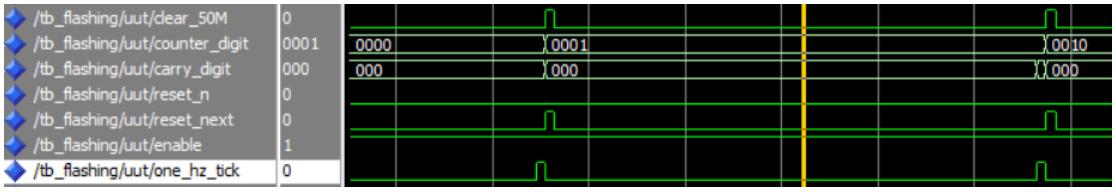


Figure 15: Zoomed waveform at a tick boundary, showing the digit counter advancing.

7.3 Results

The waveform shows the digit output changing exactly when the tick pulse occurs, confirming that clock division and modulo-10 counting work correctly together. The digit sequence 0–9 repeats without skipping or double-counting.

8 Part 5: Reaction Timer

8.1 Design Description

The reaction timer measures how quickly a user can respond to a visual cue. After reset, the system waits for a user-selected delay, lights an LED, counts elapsed milliseconds until the user presses a stop button, and holds the final result on the four seven-segment displays. The key sub-blocks are:

- a 1 kHz tick generator (clock divider);
- an 8-bit wait counter whose terminal count is set by `SW(7 downto 0)`;
- a 16-bit reaction counter that records elapsed milliseconds;
- a three-state FSM controller;
- a binary-to-BCD converter using the Double Dabble algorithm;
- four `seven_seg_dec` decoders driving `HEX3--HEX0`.

8.2 FSM Design

The controller has three states:

Table 5: Reaction timer FSM states

State	Behavior
WAITING	Increments the wait counter on each 1 ms tick; transitions to COUNTING when the count matcl
COUNTING	Turns on `LEDR0` and increments the reaction counter every millisecond; transitions to STOPPED
STOPPED	Holds the reaction counter value until reset is pressed.

Pressing `KEY0` (reset) returns the system to WAITING and clears both counters synchronously.

8.3 Timing

The 1 kHz tick is derived from the 50 MHz clock by counting 50,000 cycles:

$$\frac{50{,}000{,}000}{50{,}000} = 1{,}000 \text{ ticks/s} \;\Rightarrow\; 1 \text{ ms per tick}$$

Each increment of the reaction counter therefore represents exactly 1 ms of elapsed time. With a 16-bit counter the maximum measurable time before overflow is 65.535 s, which is more than sufficient for a human reaction test.

```
35      TYPE state_type IS (WAITING, COUNTING, STOPPED);
36      SIGNAL state : state_type := WAITING;
37
38      SIGNAL clk_div_count : unsigned(15 DOWNTO 0) := (others => '0');
39      SIGNAL one_khz_tick  : std_logic := '0';
40
41      SIGNAL wait_counter      : std_logic_vector(7 DOWNTO 0);
42      SIGNAL reaction_counter  : std_logic_vector(15 DOWNTO 0);
43
44      SIGNAL reset_pressed     : std_logic;
45      SIGNAL stop_pressed      : std_logic;
46      SIGNAL led_on            : std_logic := '0';
47
48      SIGNAL d0, d1, d2, d3 : std_logic_vector(3 DOWNTO 0);
49
50  BEGIN
51      reset_pressed <= NOT KEY0;
52      stop_pressed  <= NOT KEY3;
53      LED <= led_on;
54
55      -- 1 kHz clock divider using process
56      PROCESS(CLOCK_50)
57      BEGIN
58          IF rising_edge(CLOCK_50) THEN
59              IF reset_pressed = '1' THEN
60                  clk_div_count <= (others => '0');
61                  one_khz_tick  <= '0';
62              ELSIF clk_div_count = to_unsigned(49999, 16) THEN
63                  clk_div_count <= (others => '0');
64                  one_khz_tick  <= '1';
65              ELSE
66                  clk_div_count <= clk_div_count + 1;
67                  one_khz_tick  <= '0';
68              END IF;
69          END IF;
70      END PROCESS;
```

(a)

```
72      -- State machine process
73      PROCESS(CLOCK_50)
74      BEGIN
75          IF rising_edge(CLOCK_50) THEN
76              IF reset_pressed = '1' THEN
77                  wait_counter     <= (others => '0');
78                  reaction_counter <= (others => '0');
79                  led_on           <= '0';
80                  state            <= WAITING;
81              ELSE
82                  CASE state IS
83                      WHEN WAITING =>
84                          IF (one_khz_tick = '1' AND (unsigned(wait_counter) /= unsigned(SW)))  THEN
85                              wait_counter <= std_logic_vector(unsigned(wait_counter) + 1);
86                          ELSIF unsigned(wait_counter) = unsigned(SW) THEN
87                                  led_on <= '1';
88                                  state  <= COUNTING;
89                          END IF;
90
91                      WHEN COUNTING =>
92                          IF one_khz_tick = '1' THEN
93                              reaction_counter <= std_logic_vector(unsigned(reaction_counter) + 1);
94                          END IF;
95                          IF stop_pressed = '1' THEN
96                              led_on <= '0';
97                              state  <= STOPPED;
98                          END IF;
99
00                      WHEN STOPPED =>
01                          -- Wait for reset to return to WAITING
02                          NULL;
03                  END CASE;
04              END IF;
05          END IF;
06      END PROCESS;
```

(b)

Figure 16: VHDL source for the reaction timer FSM and counter logic.

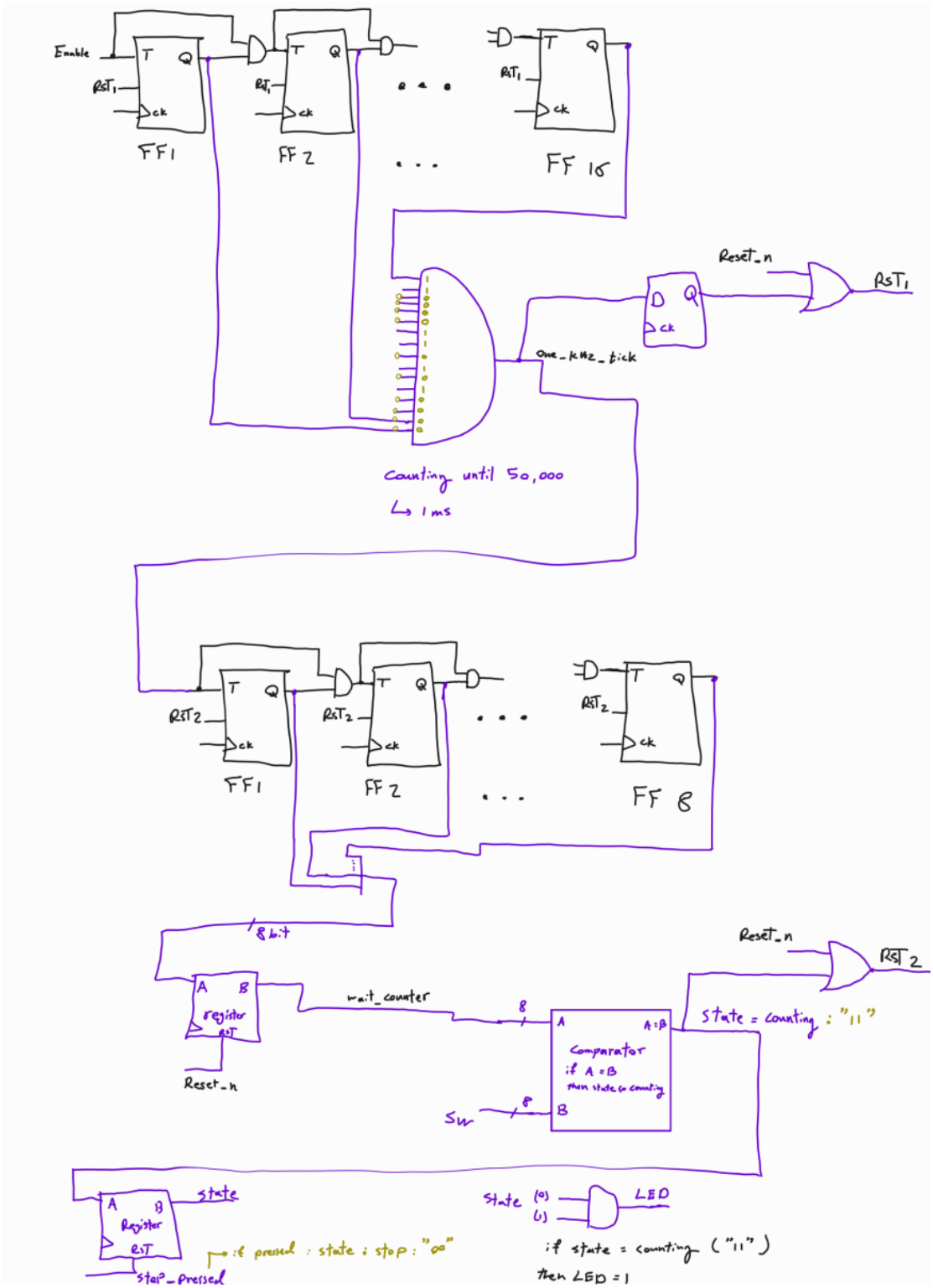


Figure 17: First section of the reaction timer block diagram (clock divider, wait counter, FSM).

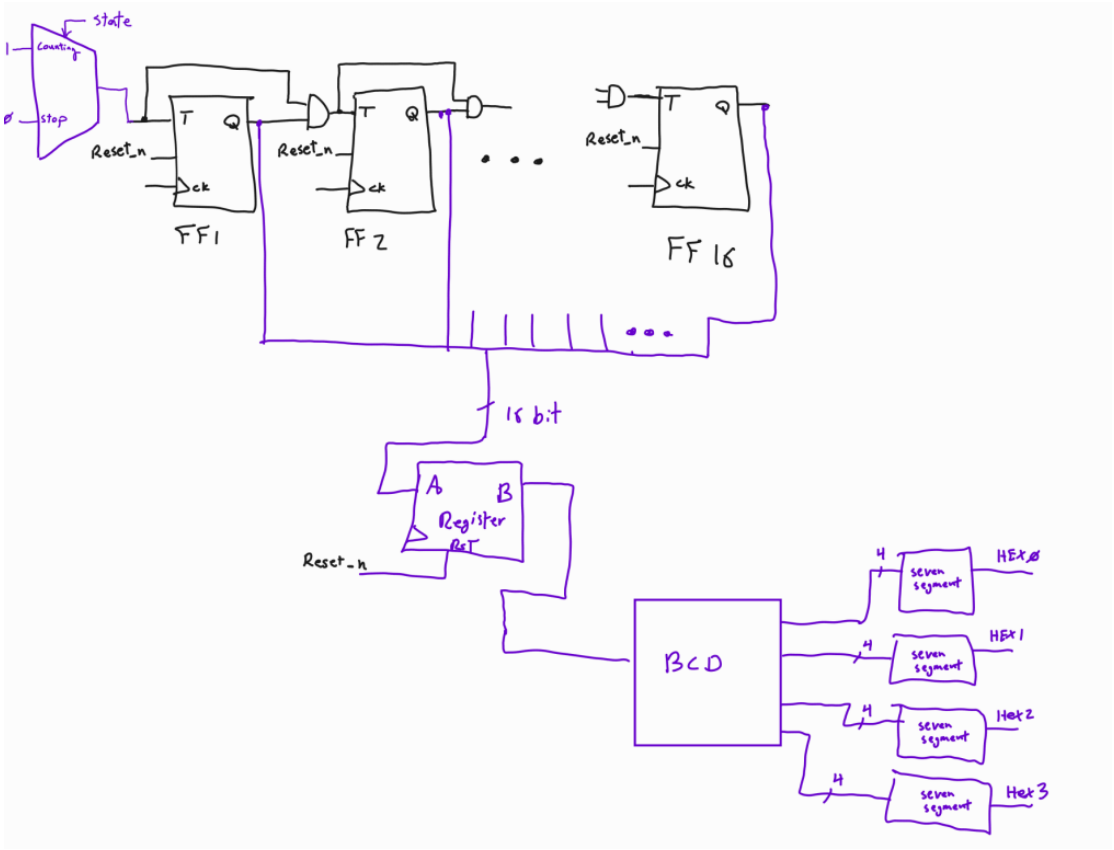


Figure 18: Second section of the block diagram (reaction counter, BCD conversion, display output).

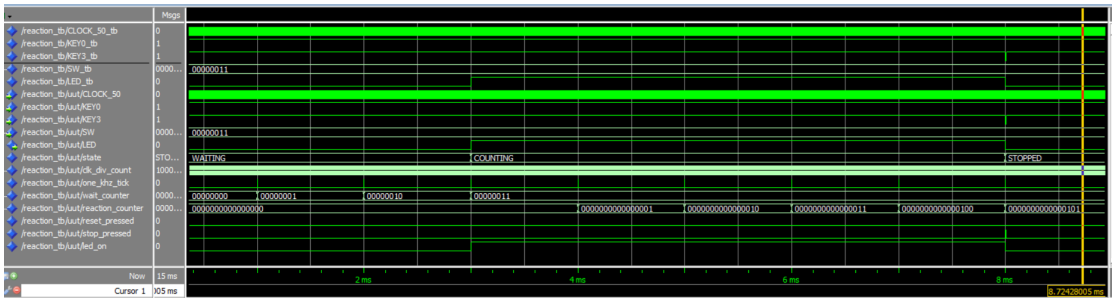


Figure 19: ModelSim waveform showing the full timer sequence.

8.4 Results

The simulation confirms the expected three-phase sequence. After reset the FSM sits in WAITING, incrementing the wait counter until it matches the switch value. The LED then turns on and the reaction counter starts. When the stop button is applied the FSM freezes the count, and the BCD output on the displays reflects the correct decimal value. The Double Dabble converter was verified separately to handle all values from 0 to 9999.

9 Verification Summary

All designs were verified by simulation before synthesis. The general strategy was to write a self-contained testbench, apply representative input sequences covering the key operating modes,

and inspect the resulting ModelSim waveform. RTL and technology views in Quartus were then used as a secondary check to confirm structural intent.

Table 6: Verification summary

Design	**Stimulus applied**	**Expected outcome**
Gated SR latch	Reset, set, hold, forbidden input	Correct latch outputs; no glitch on hold
Structural counter	Enable, pause, reset, full count cycle	Correct 16-bit binary sequence
Behavioral counter	Same as structural counter	Identical observable behavior
Digit flasher	Tick pulse, digit rollover at 9	Digits cycle 0–9 without error
Reaction timer	Delay phase, LED trigger, stop button	Final count held; correct BCD display

10 Conclusion

This project walked through a natural progression in sequential digital design, starting from a handful of NAND gates in a gated latch and finishing with a complete FSM-driven timing system. The structural designs made the hardware explicit and helped build intuition about how flip-flops, carry chains, and clock dividers actually work at the gate level. The behavioral designs then showed how the same functionality can be expressed more concisely once that intuition is in place.

Simulation was essential at every stage—both for catching bugs early and for understanding timing relationships that are easy to miss when reading code alone. Taken together, the five parts gave hands-on experience with testbench writing, waveform analysis, RTL inspection, BCD arithmetic, FSM design, and FPGA board interfacing, covering most of the core topics in sequential digital design.